库存与防超发

两桶库存 + Lua 三态 + SeedFromDB + Saga 释放 + SKU/多仓/防黄牛

gomall · 技术 deck 07

今日大纲

为什么单桶 stock 不够用

Redis Lua 三段脚本

SKU 维度库存

多仓库分布与智能选仓

启动 SeedFromDB

冷热库存分离

OrderCreate reserve

关单 release

防黄牛：单用户上限 / 设备指纹 / 邀请码

超卖应急 & 巡检对账

业务承诺与客诉答疑

为什么单桶 stock 不够用

下单到付款，中间那 15 分钟

- 用户点”提交订单”到点”立即支付”，gomall 给 15 分钟。
- 这 15 分钟里：商品**不能再被别人下单**，但也**还没真正卖出**。
- 单桶 `stock=N` 模型无法表达这个”占着但没消耗”的中间态：
  - 一减就减，用户不付钱也回不来（库存丢）；
  - 不减，并发下两个人同时下同一件（超卖）。

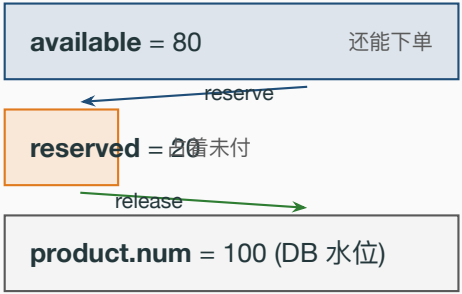
repository/cache/inventory.go:11–14 注释明确两桶定位

业务问题清单（本 deck 七连问）

1. 为什么把库存切成 **available + reserved** 两桶?
2. 抢库存为什么必须走 Redis Lua，不走 DB `SELECT FOR UPDATE`?
3. Redis 重启之后，库存数字从哪儿来?
4. 下单成功，但 DB 事务回滚，预占的库存怎么办?
5. 关单（超时 / 用户主动）触发的库存退回，落在哪一行代码?
6. 客服收到” 明明显示有货，下单却失败” 的工单，怎么回答?
7. 业务到底承诺什么——超卖能赔多少钱、误锁赔多少钱?

对应代码 *repository/cache/inventory.go* + *service/inventory/syncer.go* + *service/order.go* + *service/order_cancel.go*

两桶库存：available / reserved



两桶相加 = DB 初始水位（不计已 commit 的真扣减）。

DB 是**水位锚**，Redis 是**运行时账本**。*repository/cache/inventory.go:16–22* 两个 *key* 命名规范

物理库存 vs 可售库存：五态划分

状态	位置	含义
available	Redis 桶 1	可下单（前台” 还剩 N 件”）
reserved	Redis 桶 2	已下单未支付（占位 15min）
committed	DB `type=Paid`	已支付未发货 / 在途
in_transit	WMS 在途	物流签收前
defective	WMS 残次	不入可售

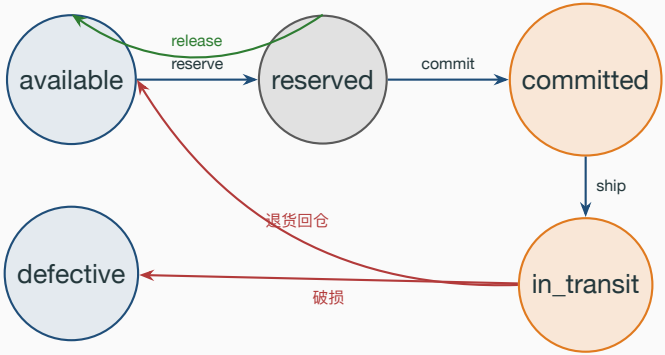
可售 = available

（前台只看这一档，绝不超卖）；**物理 = 五项之和**（财务对账盯恒等式 = product.num）。

repository/cache/inventory.go:127–138；WMS 两态归 deck 09 + 仓储管理系统

Redis Lua 三段脚本

三态生命周期（含物流回流）



- **reserve**：available -= n, reserved += n（下单时调）
- **commit**：reserved -= n（支付成功，库存真正消耗）
- **release**：reserved -= n, available += n（取消 / 关单）

in_transit / defective 归 deck 09 + WMS（仓储管理系统，本仓库未实现）。

repository/cache/inventory.go:29–66 三段 Lua 脚本完整定义

reserveScript：8 行 Lua，零超卖的全部秘密

Listing 1: *repository/cache/inventory.go:32–44 reserveScript*

```lua
local avail = redis.call('GET', KEYS[1]) -- 1. 读 available
if avail == false then
   return -2 -- 2. key 不存在 = 库存未初始化
end
local need = tonumber(ARGV[1])
if tonumber(avail) < need then
   return -1 -- 3. 不足 = ErrStockInsufficient
end
redis.call('DECRBY', KEYS[1], need) -- 4. available -= n
redis.call('INCRBY', KEYS[2], need) -- 5. reserved += n
return 1 -- 6. 成功
```

为什么必须打成一个 Lua：脚本对 Redis 是单命令，redis-server 单线程从头执行到尾，中途**不会插入**其它客户端的 GET / DECRBY / INCRBY。如果分成 3 次 RTT 做 check-then-act，500 并发立刻超卖。

commitScript / releaseScript：成对出现，对称设计

Listing 2: *repository/cache/inventory.go:48–66 commit + release*

```lua
-- commitScript：支付成功，reserved 真正消耗
local r = redis.call('GET', KEYS[1]) -- KEYS[1]=reserved
if r == false or tonumber(r) < tonumber(ARGV[1]) then
   return -1
end
redis.call('DECRBY', KEYS[1], ARGV[1]) -- reserved -= n
return 1
-- releaseScript：取消，reserved 退回 available
local r = redis.call('GET', KEYS[1]) -- KEYS[1]=reserved
if r == false or tonumber(r) < tonumber(ARGV[1]) then
   return -1
end
redis.call('DECRBY', KEYS[1], ARGV[1]) -- reserved -= n
redis.call('INCRBY', KEYS[2], ARGV[1]) -- available += n
return 1
```

commit 只动 reserved（钱已付，不退）；release 两桶都动。check $r \geq n$ 避免 reserved 跌 负

Redis Lua vs DB `SELECT FOR UPDATE`：实测尾延迟

	Redis Lua	DB 悲观锁
avg 延迟	1.24ms	1.29ms
p95 延迟	3.52ms	3.65ms
max 延迟	**136ms**	**453ms**
RPS	51,362	50,142

avg / p95 看不出差距，但 **max 拉开 3.3 倍**。原因：

- DB：BEGIN → `SELECT …FOR UPDATE` (行锁) → 业务 check → UPDATE → COMMIT，至少 4 次 RTT + 行锁串行；
- Redis Lua：**1 次 RTT**，单线程跑完 5 条命令。

stressTest/REPORT.md §6 优惠券：Redis Lua vs DB 锁

三方案对比：Redis Lua vs DB FOR UPDATE vs 分布式锁

	Redis Lua 单实例	DB 行锁	ZK / etcd 分布式锁
原子性来源	redis-server 单线程	InnoDB 行锁	ZAB / Raft 共识
单次开销	1 RTT ~ 1ms	4 RTT + 锁等待	1–3 次共识广播
水平扩展	分 slot / 分实例（业务分片）	按主键分表 / 分库	节点数加倍后写性能不增
故障恢复	AOF + SeedFromDB	主从切换 + Redo log	选主 < 10s
尾延迟	136ms（实测）	**453ms**（实测）	秒级（选主期间）
超卖风险	单实例 0(Lua 原子)	0（事务 + 锁）	0（共识 quorum）

gomall

选 **Redis Lua 单实例**：日单峰值 10w 级，单 Redis 足够；分布式锁的”选主期间不可写”对秒杀不可接受。Lua 的”单点”风险用 AOF + SeedFromDB 兜底（见 § 启动 SeedFromDB）。*stressTest/REPORT.md §6 + repository/cache/inventory.go:29–66*

反例：DB SELECT FOR UPDATE 写法（选型对照）

Listing 3: 对照写法——仓库未采用，仅做选型对照

```go
// 反面：把扣库存放进 DB 事务，悲观锁串行化
err := dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
    var p model.Product
    // 行锁 + 业务 check + UPDATE，至少 3 次 RTT
    if e := tx.Set("gorm:query_option", "FOR UPDATE").
        First(&p, productID).Error; e != nil { return e }
    if p.Num < int(num) {
        return ErrStockInsufficient
    }
    return tx.Model(&p).Update("num", p.Num-int(num)).Error
})
```

三个致命问题：(1) 表达不了 reserved 中间态；(2) 行锁串行，TP99 秒级（实测 453ms max）；(3) DB 是真理之源，热写打进 DB = 把副作用集中到最贵的资源。

零超卖的并发证据

测试：`TestInventory_NoOversellUnderConcurrency`

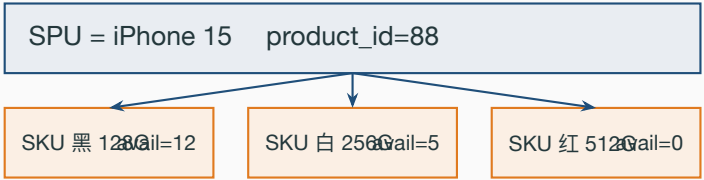
设置：`InitStock(pid, 100)`，500 goroutine 各抢 1 件。

指标	实测
成功 reserve 次数	**exactly 100**
失败次数 (ErrStockInsufficient)	400
available 终值	**0**
reserved 终值	**100**

”exactly 100” 是 `t.Fatalf("oversell: want exactly %d, got %d", initial, success)` 写死的断言：少一个少卖、多一个超卖，CI 都红。*repository/cache/inventory_test.go:52–82 NoOversellUnderConcurrency*

SKU 维度库存

同商品多规格：颜色 / 尺寸 / 套餐独立扣库存



- 当前 gomall 的 `product_id` 是 **SPU 粒度**（一个商品一行）；
- 真实 SKU 表需要 `sku_id` + `spec_json`（颜色/尺寸/套餐）；
- Redis key 改成 `stock:available:{sku_id}`，互不影响：黑色售罄不影响白色下单。

当前仓库以 *product_id* 代 *SKU*；多规格扩展见 *model/product.go* + 新增 *model/sku.go* 路线图

SKU 切粒度的影响：key 数 / 扣减粒度 / 商品页展示

维度	当前 (SPU)	SKU 扩展后
Redis key 数	1M × 2 桶 = 2M	1M × 平均 5 SKU × 2 = 10M
扣减脚本	`stock:available:{pid}`	`stock:available:{sku_id}`（同 Lua）
商品页展示	单数字	各规格独立 + 切换规格时整页 re-fetch
下单参数	{product_id, num}	{sku_id, num}（product_id 仅做检索）
关单 release	按 product_id	按 sku_id（订单表新增 sku_id 字段）

Redis 单实例承载 10M key（约 1GB 内存）问题不大；难点在订单表 schema 迁移：老订单只有 product_id，回填 sku_id 要写脚本。*model/order.go 现有 ProductID 字段；SKU 改造需迁移脚本 + 双写过渡期*

套餐 SKU：1 单同时扣 N 个底层 SKU

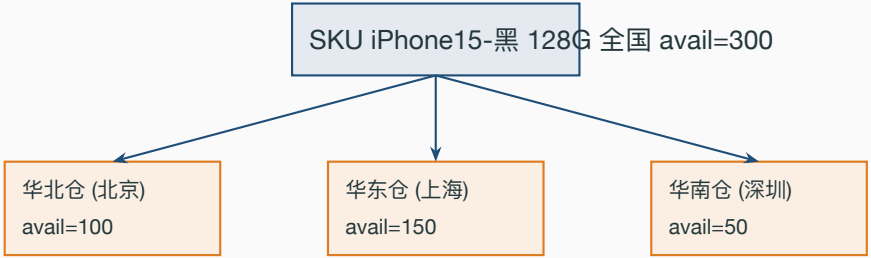
场景：手机 + 耳机 + 钢化膜套餐，下 1 单要原子扣 3 个 SKU。

- Lua 扩展为**多 KEYS 原子扣减**：
  - KEYS = [sku1_avail, sku1_resv, sku2_avail, sku2_resv, sku3_avail, sku3_resv], ARGV = [n1, n2, n3]；
  - 第一阶段循环 check 所有 avail ≥ n，任一不足返 -1；第二阶段循环 DECRBY/INCRBY 三对桶。
- Redis 单线程保证 6 个 key 的 check-act-act 不会被打断；退款 / 关单按” 套餐表” 反查所有 sku，分别 release。

当前仓库未实现套餐 *SKU*；扩展点：*repository/cache/inventory.go* 新增 *reserveBundleScript*

多仓库分布与智能选仓

库存按地域 / 仓 / 批次拆分



- Redis key 升级：`stock:available:{sku_id}:{warehouse_id}`；
- 商详页展示 = 各仓 sum；下单要先**智能选仓**再扣单仓的桶；
- 退货 / 退仓回流要带 `warehouse_id` 才能回对桶。

多仓未实现；扩展点：*model/warehouse.go* + *cache.StockAvailableKey* 加 *warehouse* 维度

智能选仓三档 + 跨仓原子性

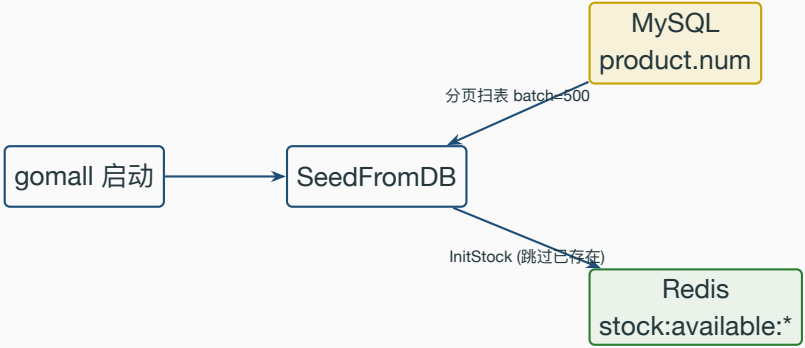
选仓策略：

1. **就近优先**：用户地址 → 省/市 → 默认仓表；不足降级第二近。
2. **凑单优先**：多 SKU 单优先选能凑齐所有 SKU 的仓，避免拆单（拆单 = 双倍物流费 + 体验差）。
3. **兜底全国仓**：就近 / 凑单都不足时走中央仓（成本高 + 时效慢，但保下单成功）。

跨仓扣减的原子性：单仓 Lua 无问题；跨仓凑单（华北 1 件 + 华东 1 件）需把所有仓 SKU key 用 hashtag `{sku_id}` 强制路由到**同一 Redis Cluster slot**，Lua 即可跨 key 原子操作。代价：单 slot 写入吞吐有上限，运维复杂度 = 1（远胜 2PC）。*Redis Cluster hashtag 规则；同 slot 才能跨 key Lua*

启动 SeedFromDB

Redis 重启之后，库存从哪里来?



- **真理之源**是 MySQL 的 `product.num`（商家后台维护，AOF / binlog 持久化）。
- Redis 只是**运行时缓存**，重启会丢，必须能从 DB 恢复。
- 但已经在跑的实例**不能**被覆盖，否则会把 reserved 状态吃掉。

initialize/inventory.go:11–17 调用入口 + *service/inventory/syncer.go:14*

SeedFromDB 主循环：分页 + EXISTS 双保险

Listing 4: *service/inventory/syncer.go:14–48 SeedFromDB*

```go
func SeedFromDB(ctx context.Context, batchSize int) error {
   if batchSize <= 0 { batchSize = 500 } // 默认页 500
   d := dao.NewDBClient(ctx); var lastID uint
   for {
      var rows []*model.Product
      q := d.Model(&model.Product{}).Order("id ASC").Limit(batchSize)
      if lastID > 0 { q = q.Where("id > ?", lastID) } // 游标分页
      if err := q.Find(&rows).Error; err != nil { return err }
      if len(rows) == 0 { break }
      for _, p := range rows {
         lastID = p.ID
         exists, _ := cache.RedisClient.Exists(ctx,
            cache.StockAvailableKey(p.ID)).Result()
         if exists == 1 { continue } // 跳过运行期数据
         cache.InitStock(ctx, p.ID, int64(p.Num)) // 覆盖写 0
      }
   }
   return nil
}
```

为什么是” 游标分页 + EXISTS 跳过”

- **游标分页**（`WHERE id > lastID LIMIT 500`）：1M 行的 product 表 `OFFSET 999500` 要扫近百万行；游标走 PK 索引，每页常数时间，启动期 DB CPU 不抖。
- **EXISTS 检查 / 跳过**（line 32–40 是关键）：
  - 冷启动：available key 不存在 → 写入 DB 水位；
  - 滚动重启 / 多副本：available 已被前一实例写过 → 跳过，**不抹平运行期 reserve**；
  - `InitStock` 是**覆盖写**（`SET k v 0`），没 EXISTS 就是事故。

service/inventory/syncer.go:34–41 EXISTS 跳过

库存预热：大促前的全量灌入

- **冷启动**（机房新部署）：SeedFromDB 跑一次，1M SKU × 500/批 = 2000 批 × 50ms ≈ 100s。
- **大促预热**（双 11 提前一天）：手动触发 `SeedFromDB(ctx, 1000)` 灌全量；抽样 1000 SKU 对比 DB num 与 Redis avail；活动前 5min 关商家改库存入口，避免双写竞态。
- **分批 + 灰度**：1000 SKU/批 + 每批 10ms sleep；先灌 1% → 10% → 100%，每档观察 5min。

service/inventory/syncer.go:14 SeedFromDB；预热可拆独立工具 cmd/inventory_seed/main.go

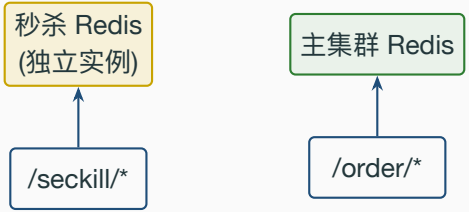
容量规划：1 万 SKU vs 100 万 SKU

	1 万 SKU	100 万 SKU
Redis key 数（两桶）	20,000	2,000,000
单 key 占用（int+overhead）	约 80B	约 80B
内存占用估算	**1.6 MB**	**160 MB** 100 万 SKU 的库存全
SeedFromDB 单批 500 耗时	50ms	50ms（DB 索引扫）
总批次	20	2,000
冷启动耗时	**1s**	**100s**

量灌入 Redis 单实例完全可承受（160MB / 16GB 实例 = 1%）。瓶颈在**冷启动时间**：100s 期间下单链路用 SeedFromDB 入口先返回 ErrStockNotInit，触发降级 banner ” 库存初始化中，请稍候”。*repository/cache/inventory.go:69–75 InitStock*；预热阶段拒单是可接受的小窗口

冷热库存分离

秒杀商品 vs 普通商品：物理隔离 Redis 实例



动机：秒杀 10w QPS 会把主集群 CPU 打满，一个慢 Lua stall 所有客户端 → 商详 / 购物车 / 下单连锁雪崩。**解法**：秒杀期间专开一个 Redis 实例，按 URL prefix 路由（`/seckill/*` 走独立 client）。

配置项	主集群	秒杀实例
持久化	AOF + RDB	**关闭**（活动结束销毁）
最大内存	16GB	2GB
TCP backlog	511	32768（接握手洪峰）

cache.RedisClientSeckill 独立 client

当前 *gomall* 共用 *Redis*；扩展点：

OrderCreate reserve

下单链路 + OrderCreate 关键片段

链路：Client → OrderCreate → ReserveStock(Lua) → DB Tx (order + outbox) → MQ delay 15min。Tx 失败 → ReleaseReservation 补偿。**先 Redis 后 DB**：DB 跑不动时 Redis 直接拒单，不淹 DB。

Listing 5: *service/order.go:58–86 reserve + tx + 补偿 release*

```go
// 1) Redis 预扣: available -> reserved, 失败直接拒单
if err = cache.ReserveStock(ctx, req.ProductID, int64(req.Num)); err != nil {
   return nil, err // ErrStockInsufficient
}
// 2) 同事务写订单 + 写 outbox
err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
   if e := dao.NewOrderDaoByDB(tx).CreateOrder(order); e != nil { return e }
   return dao.NewOutboxDaoByDB(tx).Insert(/* OrderCreated */)
})
if err != nil {
   // 3) 事务失败: 必须释放刚扣的预占 (Saga 补偿)
   if relErr := cache.ReleaseReservation(ctx,
         req.ProductID, int64(req.Num)); relErr != nil {
      util.LogrusObj.Errorf("release on tx failure: %v", relErr)
```

关单 release

15min 关单链路 + 幂等保证

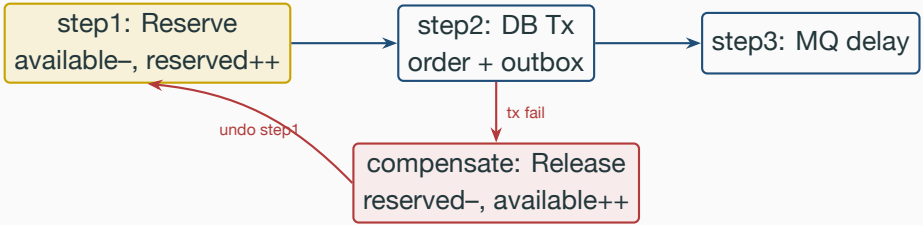
链路：MQ delay → CancelWorker → CancelUnpaidOrder → DB Tx (CloseOrderWithCheck + outbox cancelled) → ReleaseReservation（只在 closed=true 时走）。**先关单 DB 状态，后退 Redis**：先退再关时若关单失败，库存已退但订单还能支付。

Listing 6: *service/order_cancel.go:31–62 关单 + release*

```go
err = baseDao.DB.Transaction(func(tx *gorm.DB) error {
    ok, err := dao.NewOrderDaoByDB(tx).CloseOrderWithCheck(orderNum)
    if err != nil { return err }
    if !ok { return nil } // 1. 已关 -> 短路 no-op
    closed = true
    return dao.NewOutboxDaoByDB(tx).Insert(/* OrderCancelled */)
})
if err != nil { return err }
if !closed { return nil } // 2. 没真关 -> 不退库存

if relErr := cache.ReleaseReservation(ctx,
    order.ProductID, int64(order.Num)); relErr != nil {
    util.LogrusObj.Errorf("release on cancel failed: %v", relErr)
```

Saga 失败回滚：DB 落库失败必须 release



如果 step2

失败且**不补偿**：available 数字永远比真实可售数字小，长期下来库存全锁死。这就是**库存 leak** ——客户端看”无货”，DB 里其实有货。*service/order.go:79–85* 显式调用 *ReleaseReservation* 补偿

防黄牛：单用户上限 / 设备指纹 / 邀请码

黄牛攻击模型 + 四层防线

攻击套路：脚本秒下（1s 内 100 笔）+ 多账号（50 号/人）+ 多设备（模拟器农场）+ 验证码 OCR（95%+ 通过）。

防线	实现位置	防御效果
单用户上限	Redis `user:{uid}:purchased:{pid}`	同账号黄牛废
设备指纹	WebGL / Canvas / UA hash	模拟器 80% 识破
邀请码限购	大促前发券，凭码下单	池子缩到 1/10
实名认证	单证件号限购 1 件	多账号黄牛废
图灵测试	滑块 / 行为轨迹	脚本成本 +10x

防御方向：拉高黄牛

单次成本，每防一招收益率 -10%，**劝退**而非**封死**。`coupon.go:30--44` `ClaimCouponAtomic` 已实现”单用户上限”范式，库存场景套同样 Lua。

repository/cache/coupon.go:30–44；reserveScript 加 owned < perUser check

扩展 reserveScript：单用户限购上限

Listing 7: 扩展 *Lua* ——*reserveScript* + 单用户限购 *check*

```lua
-- KEYS[1]=available, KEYS[2]=reserved
-- KEYS[3]=user:{uid}:purchased:{pid}
-- ARGV[1]=n, ARGV[2]=perUserLimit
local avail = redis.call('GET', KEYS[1])
if avail == false then return -2 end
if tonumber(avail) < tonumber(ARGV[1]) then return -1 end
local owned = tonumber(redis.call('GET', KEYS[3])) or 0
if owned + tonumber(ARGV[1]) > tonumber(ARGV[2]) then
   return -3 -- 超出单人限购
end
redis.call('DECRBY', KEYS[1], ARGV[1])
redis.call('INCRBY', KEYS[2], ARGV[1])
redis.call('INCRBY', KEYS[3], ARGV[1])
redis.call('EXPIRE', KEYS[3], 86400 * 30)
return 1
```

单 Lua 加一层 check 不破坏原子性；-3 由 Go 端映射成 `ErrPerUserExceed`。

超卖应急 & 巡检对账

超卖应急流程：从发现到赔付的 4 小时

时间窗	动作	责任方
T+0 min	监控告警：可发货量 < 已支付订单量	SRE on-call
T+5 min	锁定问题 SKU：停止商详页下单入口	SRE 改路由规则
T+15 min	跑**对账脚本**：计算超卖量、影响订单数	DBA + 库存 owner
T+30 min	客服话术下发：哪些订单可以发货 / 需要赔付	客服中心
T+1h	给被超卖用户发**补偿券 + 致歉短信**	营销 + 客服
T+4h	复盘报告 + 修复脚本上线	库存 owner

关键指标：**T+5 min 内能锁单**是底线。商详页”下单入口”必须是动态开关，不能等代码发布。*应急 SOP；对*

对账脚本 + 客服补偿话术

恒等式：`available + reserved + committed = product.num`（前两项来自 `GetStockSnapshot`，committed = DB `SUM(num) WHERE type=Paid`）。

- **巡检阈值**：偏差 < 1% 正常；1–5% 钉钉 P3；> 5% 电话 P1；大促前强校验 > 0.1% 拦截活动开始。
- **校验时机**：10min 抽样 100 个热门 SKU + 凌晨 3:00 全量对账。
- **修复**：SRE 用 `InitStock(pid, 真实可售)` 人工覆盖（误锁可接受、超卖不可）。

客服补偿档位（按发现时间）：< 24h = 退款 + 10 元券；24–72h = 退款 + 30 元券 + 致歉短信；> 72h = 退款 + 50 元券 + 同档替代 + 1v1 客服。极端事件 24h 内 APP 首页公告 + 公开复盘。*repository/cache/inventory.go:127–138 GetStockSnapshot*；赔付额度由风控 + 客服中心定

业务承诺与客诉答疑

业务承诺：超卖 vs 误锁，两条不同的红线

事故	用户体感	业务代价
超卖	下单成功，无法发货	**真金白银赔** + 商誉
误锁	显示”已售罄”	转化率轻微下降

- 超卖 = 法律 / 合同事件，必须**零容忍**（NoOversell 单测就是这一条的工程化）。
- 误锁 = 体验事件，可以**容忍一阵**（高峰过后 / 关单超时后会自愈）。
- 这个不对称是**所有设计选择**的源头：选 Redis 单线程、选先 reserve 后 commit、选 EXISTS 跳过 …

repository/cache/inventory.go:11–14 注释揭示这条业务红线

客服话术：50001 vs 70001 vs 70002

状态码	触发位置	客服话术
50001	ReserveStock 返回 ErrStockInsufficient	” 本商品已售罄”
70001	全局 / 秒杀限流命中	” 活动太火爆，请稍后”
70002	支付熔断器 Open	” 支付服务暂忙，1 分钟后”
60002	同 key 重复下单	” 您已下单成功”

50001 和 70001 都是” 拒单”，但根因不同：

- 50001 是**真的**没货（available = 0），重试也没用；
- 70001 是**速率**被限，等几秒重试就行。

pkg/e/code.go 业务码定义；*50001* 在 *deck 10* 业务码总表中作为库存不足语义

典型客诉：明明显示有货，下单却失败

Q：商详页显示库存 50，我下单却提示” 已售罄”?

两种根因：

- **商详页缓存 vs Redis available 不一致**：
  - 商详页用 5min 商品缓存（deck 03），num 字段是 5 分钟前的快照；
  - 高峰期，available 已经从 50 跌到 0，但商详页还显示 50；
  - 应对：商详页改成显示” 是否有货” boolean，或缓存 TTL 调到 30s。
- **库存被预占未释放**：
  - 大量未支付订单堆在 reserved 桶，available = 0；
  - 15 min 后被关单 worker 释放回去；
  - 应对：关单 worker 心跳监控，延迟超过 2min 报警。

service/order_cancel.go:59 release 调用是这条客诉自愈的关键

面试 Q&A (1/3)：基础设计 + 容灾

Q1：为什么不能用单条 `UPDATE product SET num=num-1 WHERE num>0`？

A：单 SQL 防得了超卖，但表达不了”占着未付”。下单到支付 15min，单桶只能选”立刻扣”（不付就丢）或”不扣”（并发超卖），都不可接受。代码：`inventory.go:11--14`。

Q2：Redis 挂了 30 秒，恢复后库存怎么办？

A：AOF / RDB 保住大头；丢失部分由 SeedFromDB 重启时从 `product.num` 回灌；运行期 reserved 通过 EXISTS 跳过保护；差额走巡检脚本。代码：`syncer.go:14--48`。

Q3：DB 事务成功但 Redis commit 失败怎么办？

A：outbox 事件落库后消费者重试 CommitReservation；reserved 偏高由巡检 leak 报警接管。属”业务可接受 + 监控可发现”的取舍。代码：`service/order.go:64--78`。

Q4：100 万 SKU 启动要多久？内存多少？

A：每 SKU 两桶 × 80B = 160 MB（16GB 实例 1%）；冷启动 500/批 × 50ms = 100s。期间下单返 ErrStockNotInit 触发降级 banner。

面试 Q&A (2/3)：SKU / 多仓 / 选型 / 秒杀

Q5：多规格（颜色、尺寸）怎么扩展?

A：Redis key 从 `stock:available:{pid}` 升级到 `{sku_id}`，Lua 一字不改；难点在订单表迁移（老订单只有 product_id），需双写过渡期 + 回填脚本。

Q6：跨仓凑单怎么保原子性?

A：所有仓 SKU 的 key 用 Cluster hashtag `{sku_id}` 强制路由到同一 slot，Lua 同 slot 内原子操作多 key。代价：单 slot 写入吞吐受限，运维复杂度 = 1（远胜 2PC）。

Q7：为什么不上 ZK / etcd 分布式锁?

A：分布式锁基于共识（ZAB / Raft），单次 1–3 次广播比 Lua 慢一个数量级；选主期间（5–10s）不可写，秒杀不可接受。日单峰 10w 级单 Redis 足够。论证：`stressTest/REPORT.md §6`。

Q8：秒杀为什么要分 Redis 实例?

A：Redis 单线程，一个慢 Lua stall 所有客户端。秒杀 10w QPS 会把主集群 CPU 占满，影响商详 / 购物车 / 下单。秒杀实例关 AOF（结束销毁）+ TCP backlog 32768，吞吐翻倍。

面试 Q&A (3/3)：防黄牛 / 应急 / 巡检 / 并发证据

Q9：怎么防黄牛批量下单？

A：四层防线——单用户上限（Lua 加 `owned < perUser` check）+ 设备指纹 + 邀请码限购 + 实名认证。每层把黄牛成本拉高 10%，劝退而非封死。模板：`coupon.go:30--44 ClaimCouponAtomic`。

Q10：发生超卖了，怎么应急？

A：4 小时应急 SOP ——T+5min 锁单（关商详页下单入口）、T+15min 跑对账脚本定位范围、T+30min 发客服话术、T+1h 发补偿券。技术侧关键：**下单入口要动态可关**，不能等发版。

Q11：怎么做库存巡检？

A：恒等式 `available + reserved + committed = product.num`，前两项来自 `GetStockSnapshot`，committed = `SUM(num) WHERE type=Paid`。偏差 1% P3、5% P1。代码：`inventory.go:127--138`。

Q12：500 并发抢 100 件，怎么证明真不超卖？

A：`TestInventory_NoOversellUnderConcurrency` 写死 `success == exactly 100`，少

代码位置一览

实现

reserve / commit / release Lua *repository/cache/inventory.go:32–66*
InitStock + Snapshot *repository/cache/inventory.go:69–138*
SeedFromDB + 启动注入 *service/inventory/syncer.go:14–48；initialize/inventory.go:11–17*
OrderCreate reserve + 补偿 *service/order.go:57–86*
CancelUnpaidOrder release *service/order_cancel.go:14–63*

测试 & 压测

Reserve / Release / NoOversell *repository/cache/inventory_test.go:10–82*
Lua vs DB 尾延迟 *stressTest/REPORT.md §6*

配套业务 deck：10-traffic-governance.tex / 11-consistency.tex